

Deep Learning

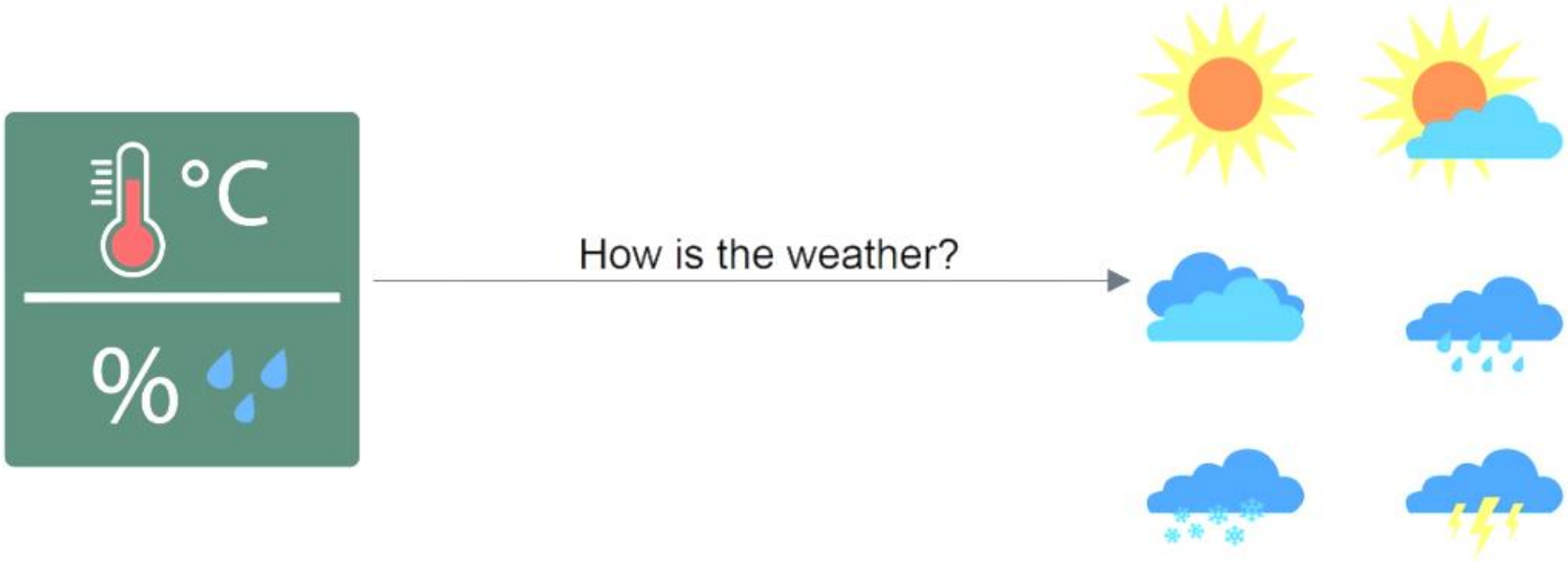
Hindriyanto D. Purnomo, Ph.D

Recurrent Neural Network

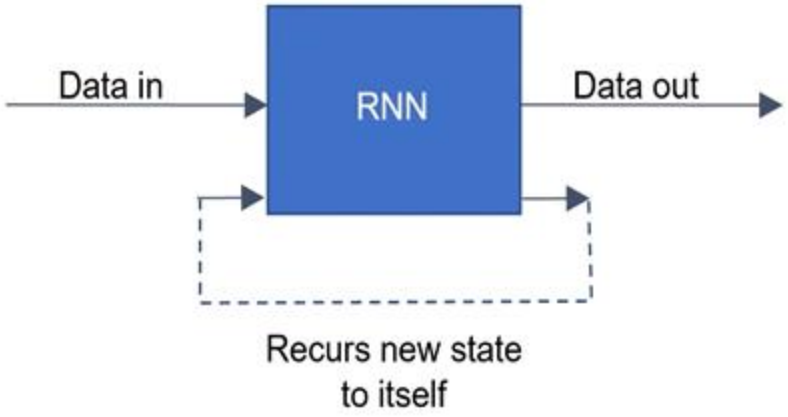
Recurrent Neural Network

- Sequential Data – data dengan dependensi
- Time Series – pengamatan dari waktu ke waktu
- Banyak datasets bersifat sequential
- Data sequensial tidak bisa ditangani dengan baik oleh Neural Networks standar/tradisional

Recurrent Neural Network

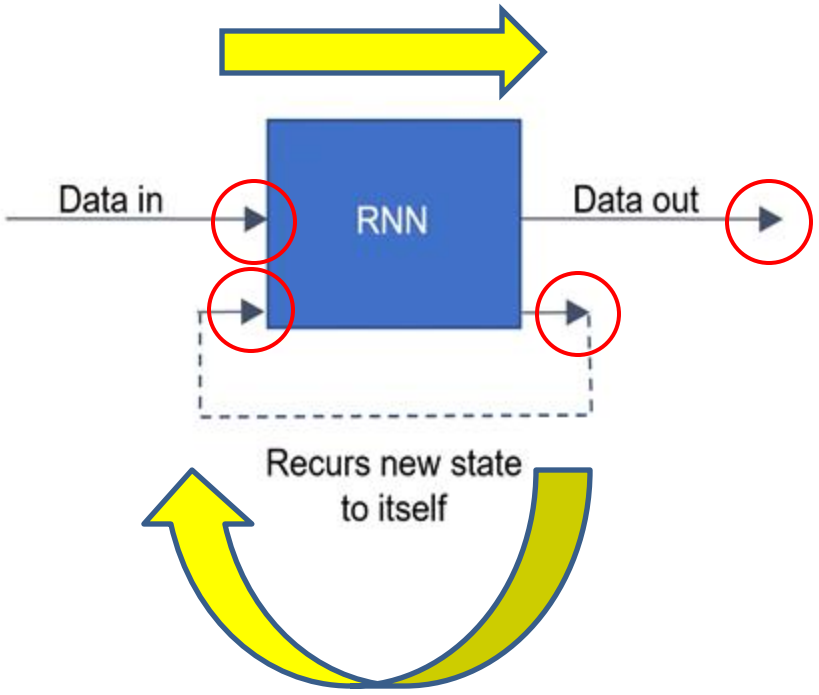


Recurrent Neural Network



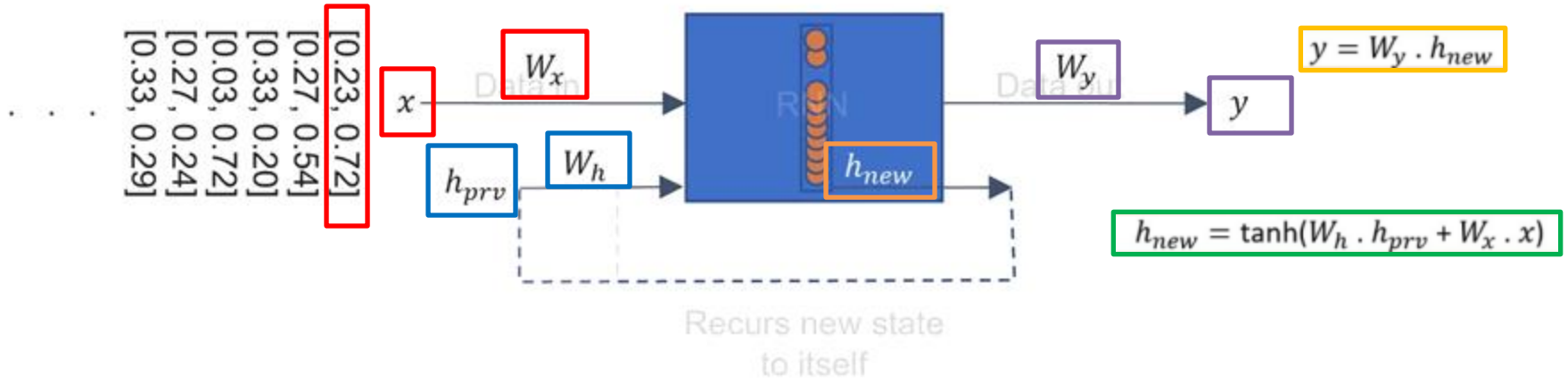
- Sebuah *Recurrent Neural Network* (RNN) cocok digunakan untuk data sekuensial
- RNN menggunakan *state* untuk merekam output pada setiap tahap yang dilalui. Dengan kata lain, state berfungsi sebagai memori bagi RNN
- Secara rekursif, state kembali diinput ke dalam RNN, bersama data yang berikutnya

Recurrent Neural Network



- Input RNN terbagi 2 yaitu:
  - Input data point
  - Input state sebelumnya
- Proses pada RNN menghasilkan 2 output yaitu:
  - Output RNN
  - Output current state (hidden)
- Proses pada RNN terus berlangsung secara rekursif sampai semua data terinput

Recurrent Neural Network



Input:

- x : data baru, W_x : bobot untuk x
- h_{prv} : state sebelumnnya, W_h : bobot untuk h_{prv}

Output:

- h_{new} : state baru (menjadi h_{prv} pada rekursi selanjutnya)
- y : network output, W_y : bobot dari y

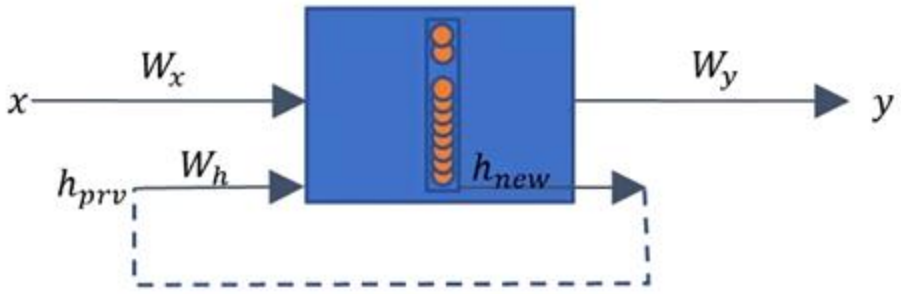
Recurrent Neural Network

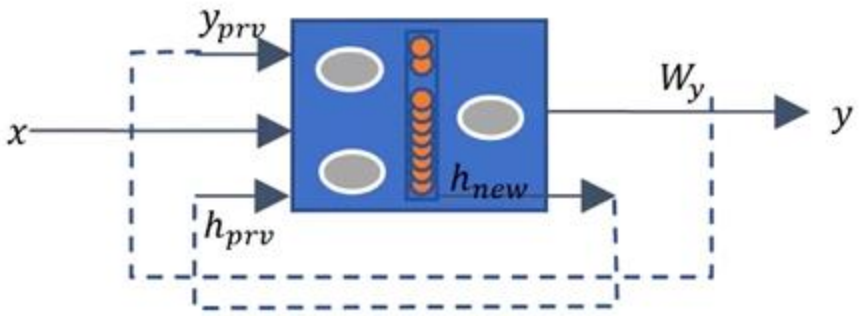
- Network harus mengingat semua state pada semua titik waktu
  - Akibat: Boros dalam komputasi dan memori
  - Solusi: Hanya menyimpan state dalam jendela waktu tertentu saja
- Sensitif terhadap perubahan parameter
  - Akibat: Vanishing gradient, Exploding gradient
  - Solusi: Long Short-Term Memory (LSTM)

Model LSTM

RNN memiliki kelemahan yaitu:

- Harus merekam semua state (boros komputasi dan memori)
- Vanishing gradient (datar)
- Exploding gradient
- Akibatnya RNN tidak dapat menangani long sequences dengan baik.
- Solusi: model Long Short-Term Memory (LSTM)

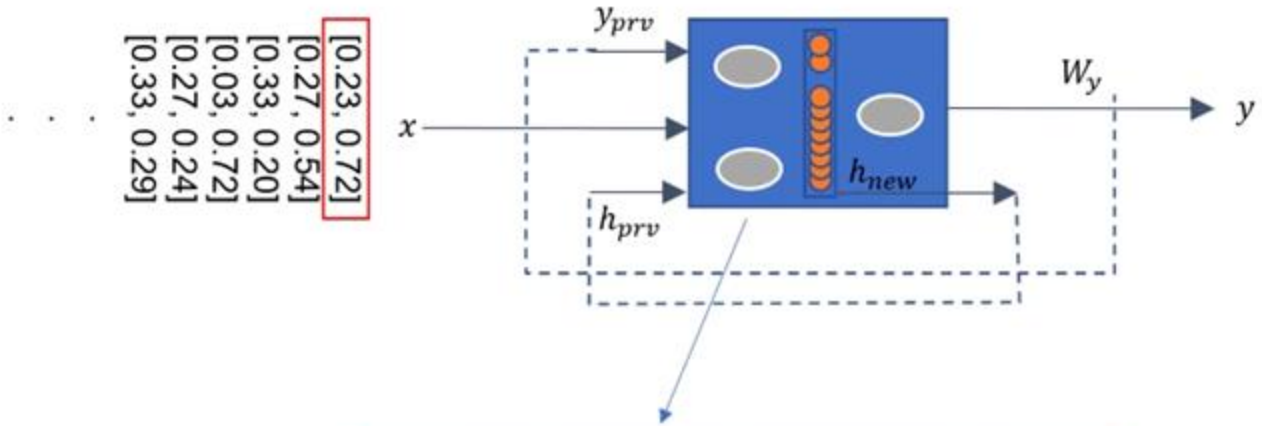




Model LSTM

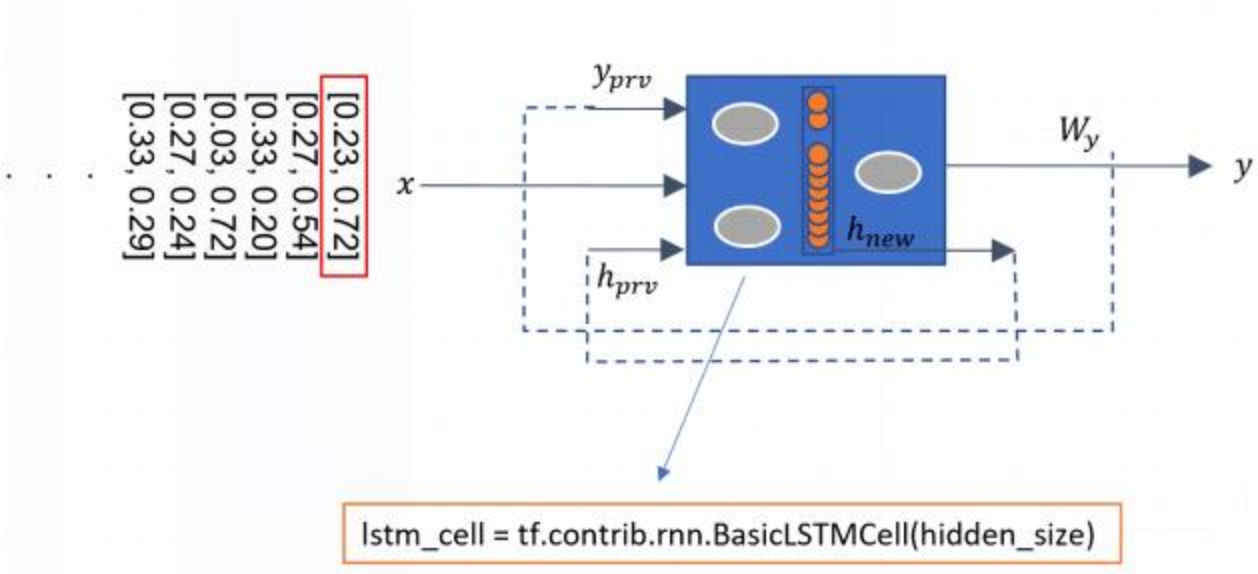
Sebuah LSTM unit terdiri dari:

1. memory cell: untuk menampung data
2. logistic gates:
   - write gate: untuk menuliskan data ke memory cell
   - read gate: membaca data dan mengirimkan kembali ke RNN
   - forget gate: menentukan apakah data dihapus atau dipertahankan



Model LSTM

- Dengan memanipulasi ketiga gate tersebut, RNN mampu mengingat data yang masih dibutuhkan dan menghapus data yang sudah tidak dibutuhkan



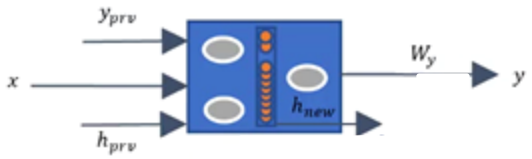
Data Flow pada LSTM

Time step 1: Input data pertama, menghasilkan hidden state dan RNN output.

Time step = 1

. . .

[0.33, 0.29]
[0.27, 0.24]
[0.03, 0.72]
[0.33, 0.20]
[0.27, 0.54]
[0.23, 0.72]



Time step = 2

Time step = 3

Data Flow pada LSTM

Time step 2: previous hiddden state dan output RNN diinput kembali bersama data berikutnya, menghasilkan output hidden state baru dan output RNN baru

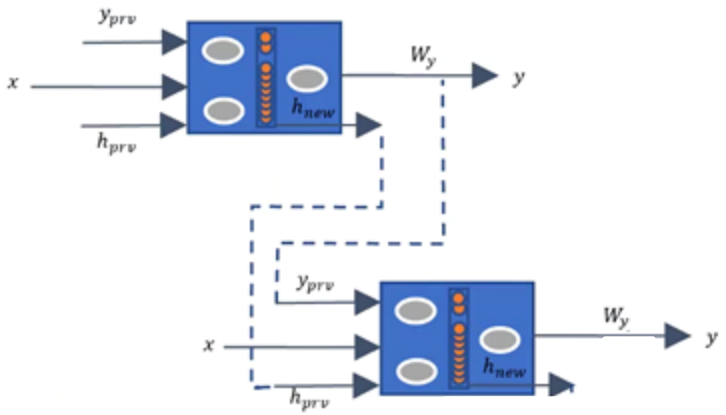
.

Time step = 1

Time step = 2

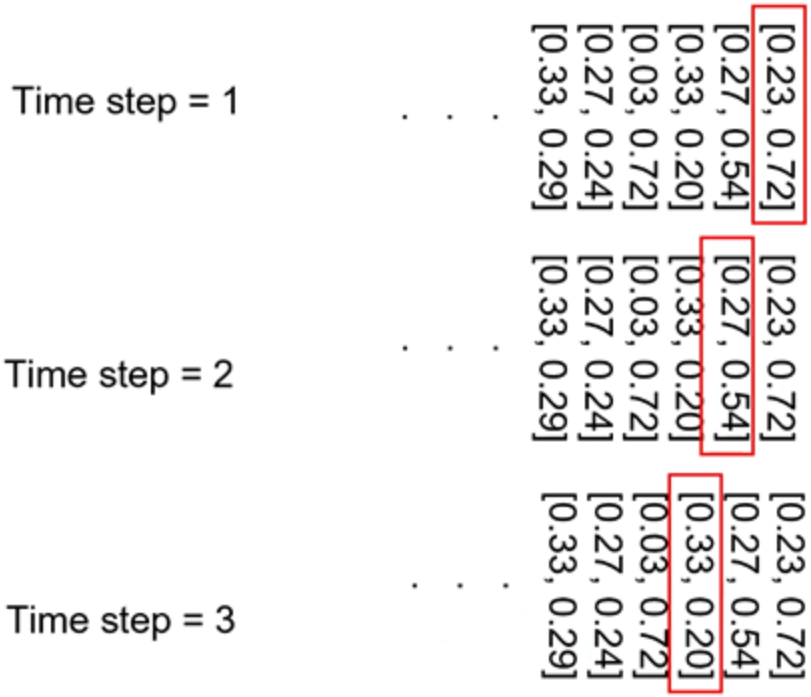
Time step = 3

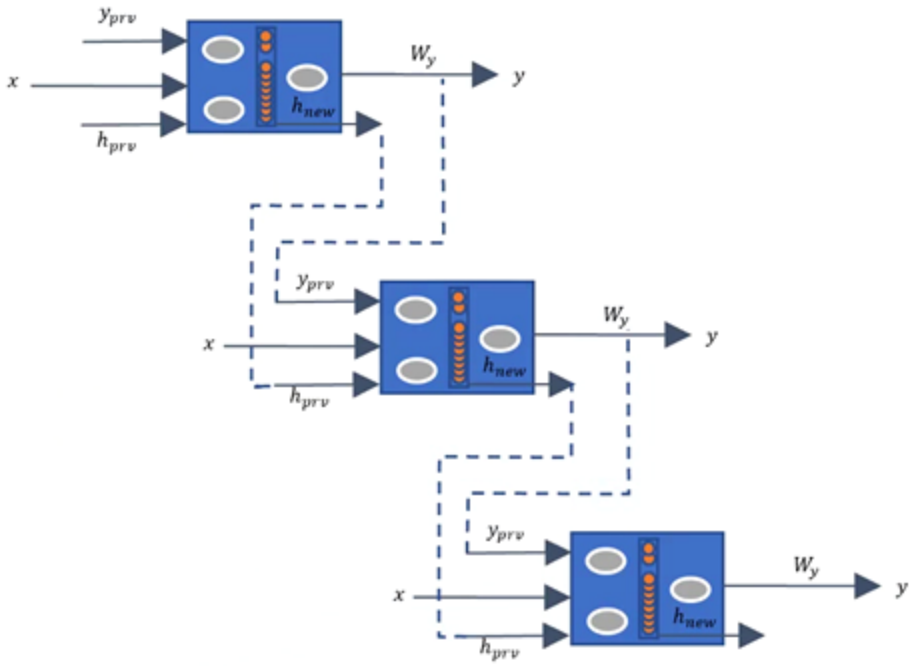
[0.23, 0.72] [0.23, 0.72]
[0.27, 0.54] [0.27, 0.54]
[0.33, 0.20] [0.33, 0.20]
[0.03, 0.72] [0.03, 0.72]
[0.27, 0.24] [0.27, 0.24]
[0.33, 0.29] [0.33, 0.29]



Data Flow pada LSTM

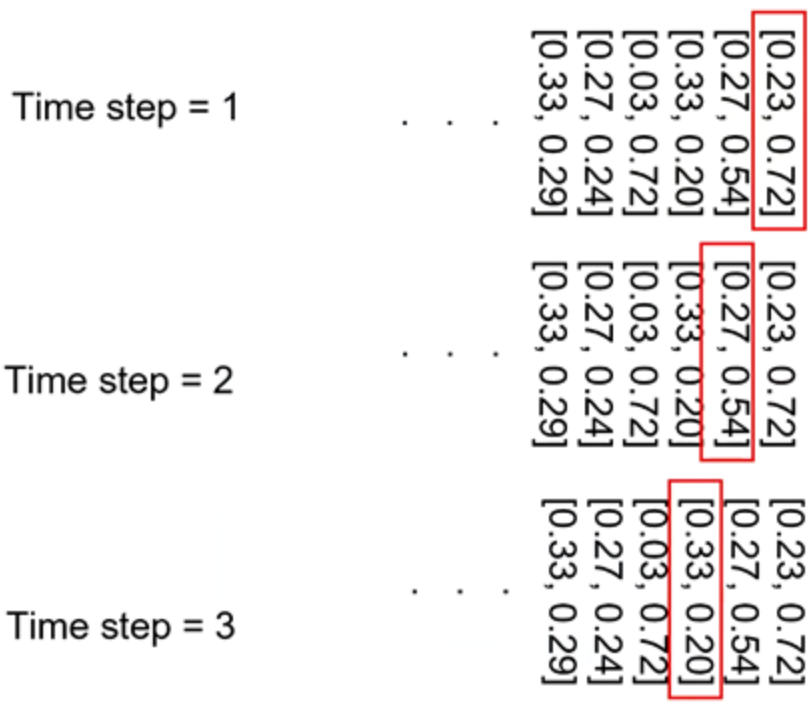
...demikian seterusnya pada time step berikutnya sampai seluruh data terinput.

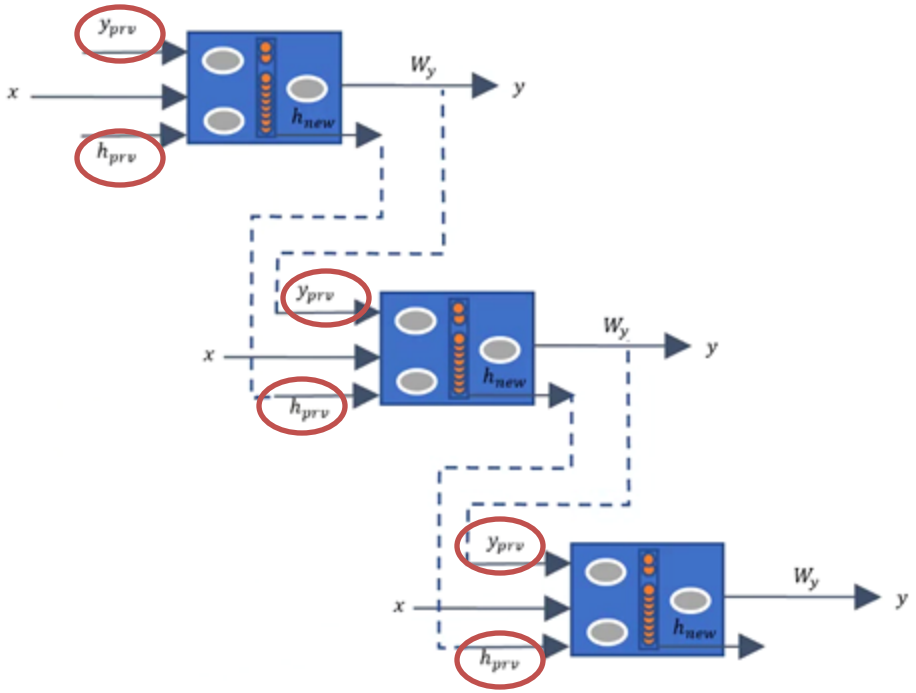


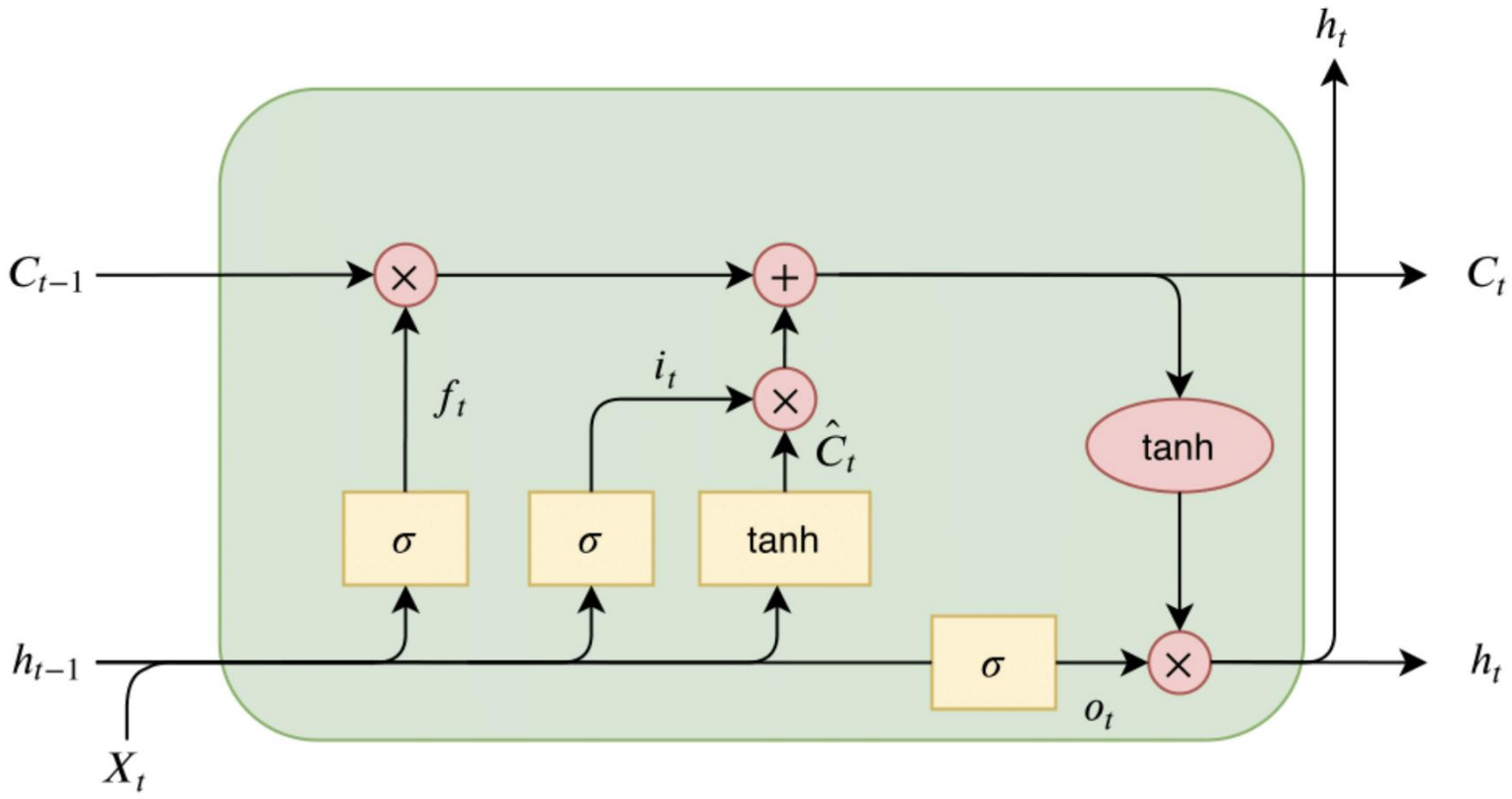


Data Flow pada LSTM

- Hidden state
- Previous output

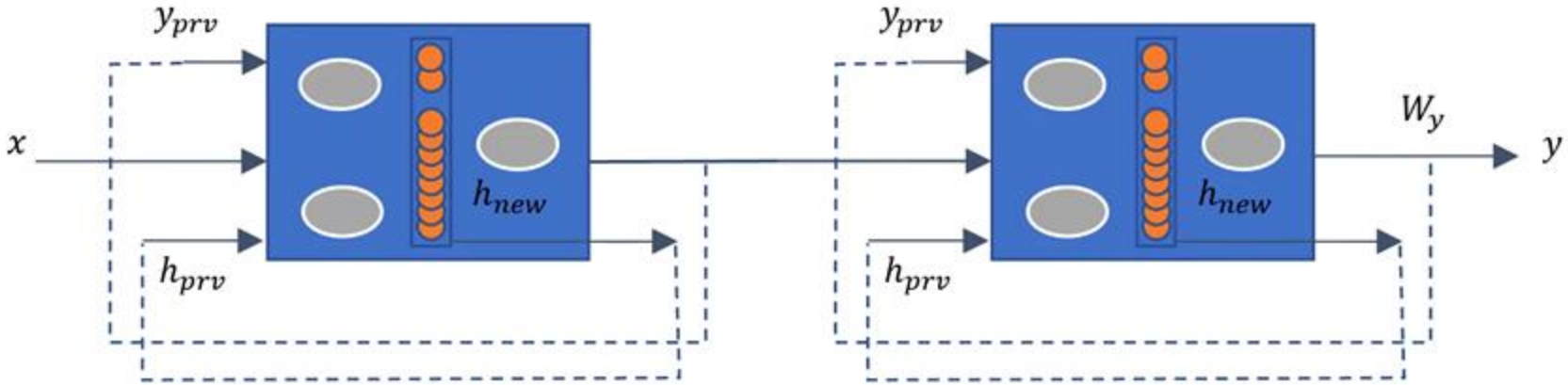





h_t
C_{t-1}
C_t
f_t
i_t
$\hat{C}_t$
tanh
σ
σ
tanh
h_{t-1}
σ
h_t
o_t
X_t

Stacked LSTM

Stacked LSTM membuat model mampu mengekstraksi fitur dengan lebih baik sehingga meningkatkan akurasi



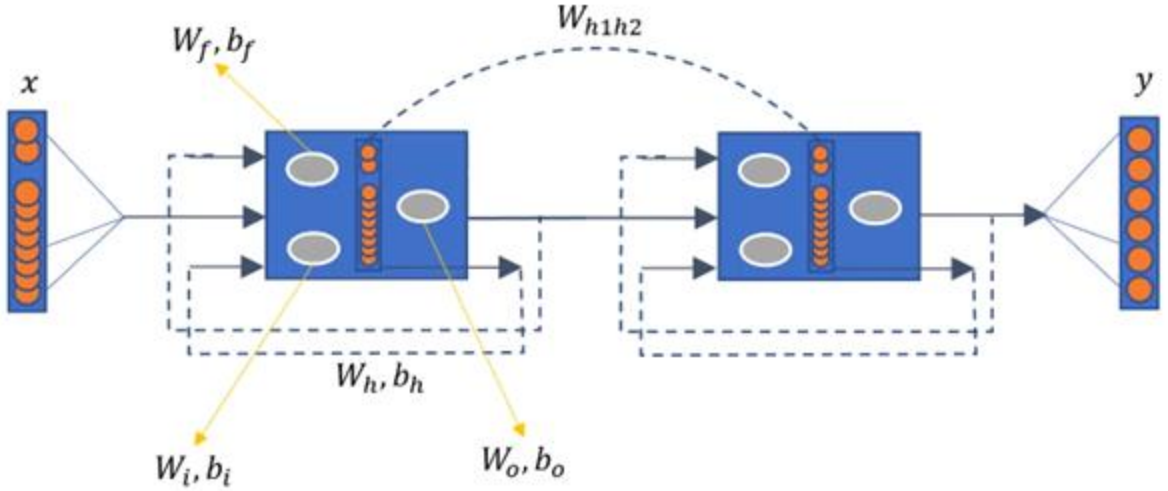

```
lstm_cell = tf.contrib.rnn.BasicLSTMCell(hidden_size)
```

```
stacked_lstm = tf.contrib.rnn.MultiRNNCell([lstm_cell] * 2)

output, state = tf.nn.dynamic_rnn(stacked_lstm, input_data)
```

Stacked LSTM

- Melalui forget gate, model mempelajari seberapa jauh periode data ke belakang yang digunakan (di luar itu data dikeluarkan dari proses).
- Input gate menginput data baru (x) dengan bobot W_i dan bias b_i
- Read gate membaca output RNN dan menginput kembali dengan bobot W_f dan bias b_f
- Hidden state diinput kembali dengan bobot W_h dan b_h



Terima kasih